

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: **COMPUTER NETWORKS**
COURSE CODE: **CSA0708**

NAME : S DHANUSH
REG NO : 192524317

COURSE OUTCOME COVERED

CO5: Measure network performance using key metrics with simulation tools. (BL4,BL5)

Assessment Tool Weightage: 30%

Annexure A

Pseudocode Writing Task (Algorithm Design)

1. In a network simulation using Cisco Packet Tracer, a student records total data received as 8000 bits in 4 seconds and observes that 200 packets were sent while 180 packets were received. Write a pseudocode algorithm to calculate the network throughput in bits per second and the packet loss percentage based on the given values. The algorithm should include validation to ensure that time and total packets sent are not zero to avoid errors. It should display both the throughput and packet loss results clearly. Add logic to classify the network as Efficient if throughput is greater than 1500 bps and packet loss is less than 10%, otherwise classify it as Inefficient. Finally, ensure the algorithm outputs a complete summary of the network performance based on the calculated metrics.
2. An organization has multiple network links connecting its branches. The administrator wants to monitor bandwidth utilization and identify overloaded links. Write a pseudocode algorithm that periodically collects the bandwidth usage of each network link and stores the values in an array or list. Use a loop to continuously monitor all links. Add conditions to detect when bandwidth utilization exceeds 80% and recommend switching traffic to a backup link. Explain how this algorithm supports efficient bandwidth management and prevents network congestion.
3. A company has six branch offices connected through multiple network links. Each link has different bandwidth, delay, and utilization values, which change throughout the day. The network administrator wants to continuously identify the best communication path between the headquarters and each branch based on overall link quality rather than just the shortest distance. Write a pseudocode algorithm that periodically collects the bandwidth, delay, and utilization of every network link and stores the values in suitable arrays or lists. Use a loop to

repeat the monitoring process at regular intervals. Calculate a link quality score for each path using the collected metrics and select the path with the highest score. Include conditions to detect when a link becomes overloaded or fails, automatically selecting an alternate path and generating an alert for the administrator. Explain how your algorithm improves network reliability, routing efficiency, and fault tolerance while adapting to changing network conditions.

RUBRICS FOR CALCULATION					
Criteria	Weight ag e	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understan din g	Incorrect interpreta tio n
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/m od ules	Minor inefficien cie s	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonabl e efficienc y	Handles basic cases only	Incomple te or inefficient

ANSWERS

1. Throughput and Packet Loss

BEGIN

```
// Store the total amount of data received in bits
totalDataReceived ← 8000
```

```
// Store the time taken in seconds
```

```

timeTaken ← 4

// Store the total number of packets sent
packetsSent ← 200

// Store the total number of packets received
packetsReceived ← 180

// Check whether time or packets sent are zero or negative
// This avoids division by zero errors
IF timeTaken <= 0 OR packetsSent <= 0 THEN

    DISPLAY "Error: Time and packets sent must be greater than zero"

ELSE

    // Calculate throughput using:
    // Throughput = Total Data Received / Time Taken
    throughput ← totalDataReceived / timeTaken

    // Calculate the number of lost packets
    packetsLost ← packetsSent - packetsReceived

    // Calculate packet loss percentage using:
    // Packet Loss % = (Packets Lost / Packets Sent) × 100
    packetLossPercentage ← (packetsLost / packetsSent) * 100

    // Display the calculated network values
    DISPLAY "Throughput: ", throughput, " bps"
    DISPLAY "Packet Loss: ", packetLossPercentage, "%"

    // Check network performance
    // Network is efficient only if throughput is greater than
    // 1500 bps AND packet loss is less than 10%
    IF throughput > 1500 AND packetLossPercentage < 10 THEN

        DISPLAY "Network Status: EFFICIENT"

    ELSE

        DISPLAY "Network Status: INEFFICIENT"

    END IF

    // Display the complete network performance summary
    DISPLAY "----- NETWORK PERFORMANCE SUMMARY ---- "
    DISPLAY "Total Data Received: ", totalDataReceived, " bits"
    DISPLAY "Time Taken: ", timeTaken, " seconds"

```

```

        DISPLAY "Packets Sent: ", packetsSent
        DISPLAY "Packets Received: ", packetsReceived
        DISPLAY "Throughput: ", throughput, " bps"
        DISPLAY "Packet Loss: ", packetLossPercentage, "%"

    END IF

END

```

Explanation:

The above algorithm calculates the network throughput and packet loss percentage. First, it validates that the time taken and packets sent are greater than zero to avoid division errors. Then, throughput is calculated by dividing the total data received by the time taken. Packet loss is calculated using the difference between packets sent and received. Finally, the algorithm checks whether the throughput is greater than 1500 bps and packet loss is less than 10%. Based on these conditions, the network is classified as Efficient or Inefficient and a complete performance summary is displayed.

2. Bandwidth Utilization Monitoring

```

BEGIN

    // Get the total number of network links
    INPUT numberOfLinks

    // Create an array to store bandwidth usage
    // of each network link
    CREATE array bandwidthUsage[numberOfLinks]

    // Continuously monitor the network links
    WHILE TRUE DO

        // Check each link one by one
        FOR i ← 0 TO numberOfLinks - 1 DO

            // Collect the current bandwidth utilization
            // and store it in the array
            INPUT bandwidthUsage[i]

            // Check whether the bandwidth utilization
            // is greater than 80%
            IF bandwidthUsage[i] > 80 THEN

                // Link is overloaded
                DISPLAY "WARNING: Link ", i + 1, " is overloaded"

                // Recommend moving traffic to another link
                DISPLAY "Recommendation: Switch traffic to backup link"
            END IF
        END FOR
    END WHILE
END

```

```

ELSE

    // Link utilization is within the safe limit
    DISPLAY "Link ", i + 1, " is operating normally"

END IF

END FOR

// Display that all links have been checked
DISPLAY "All network links monitored"

// Wait for a specified time before checking again
WAIT for specified monitoring interval

END WHILE

END

```

Explanation:

The above algorithm continuously monitors the bandwidth utilization of all network links. The utilization value of each link is stored in an array and checked using a loop. If the bandwidth utilization exceeds **80%**, the link is identified as overloaded and the algorithm recommends switching traffic to a backup link. Otherwise, the link is considered to be operating normally. This process is repeated at regular intervals, which helps in detecting congestion early and managing bandwidth efficiently.

3. Best Path Selection Based on Link Quality

```

BEGIN

    // The company has six branch offices
    numberOfBranches ← 6

    // Create arrays to store network information
    // for every link
    CREATE bandwidth[]
    CREATE delay[]
    CREATE utilization[]
    CREATE linkStatus[]
    CREATE qualityScore[]

    // Continuously monitor the network
    WHILE TRUE DO

        DISPLAY "NETWORK MONITORING STARTED"

        // Check every available network link
        FOR each network link DO

```

```
// Collect the available bandwidth of the link  
COLLECT bandwidth
```

```
// Collect the delay of the link  
COLLECT delay
```

```
// Collect the current utilization percentage  
COLLECT utilization
```

```
// Check whether the link is working or failed  
COLLECT linkStatus
```

```
// Store all collected values in arrays  
STORE bandwidth  
STORE delay  
STORE utilization  
STORE linkStatus
```

```
// Check whether the network link has failed  
IF linkStatus = "FAILED" THEN
```

```
    // Failed links should not be selected,  
    // so their quality score is set to zero  
    qualityScore  $\leftarrow$  0
```

```
    // Generate an alert for the administrator  
    DISPLAY "ALERT: Link failure detected"
```

```
    // Select another available path  
    SELECT alternate path
```

```
// Check whether the link is overloaded  
ELSE IF utilization > 80 THEN
```

```
    // Generate overload warning  
    DISPLAY "ALERT: Link is overloaded"
```

```
    // Select another path to avoid congestion  
    SELECT alternate path
```

ELSE

```
// Calculate the link quality score
// Higher bandwidth gives a better score
// Lower delay gives a better score
// Lower utilization gives a better score
qualityScore ←
    (bandwidth * 0.5)
    - (delay * 0.3)
    - (utilization * 0.2)
```

END IF

END FOR

```
// Check the best path for each branch office
FOR branch ← 1 TO numberOfBranches DO
```

```
// Select the path having the highest quality score
bestPath ← path with highest qualityScore
```

```
// Check whether a suitable path is available
IF bestPath is available THEN
```

```
// Use the best quality path
SELECT bestPath
```

```
// Display the selected path
DISPLAY "Best path selected for Branch ", branch
```

```
// Display its quality score
DISPLAY "Quality Score: ", highest qualityScore
```

ELSE

```
// If no path is available, generate an alert
DISPLAY "ALERT: No suitable path available"
```

END IF

END FOR

```
// Monitoring cycle is completed
DISPLAY "Monitoring completed"
```

```
// Wait for some time and repeat monitoring  
WAIT for specified monitoring interval
```

```
END WHILE
```

```
END
```

Explanation:

The above algorithm continuously monitors the bandwidth, delay, utilization, and status of every network link. These values are used to calculate a link quality score, where higher bandwidth and lower delay and utilization provide better performance. The path with the highest quality score is selected as the best path for communication. If a link becomes overloaded or fails, the algorithm automatically selects an alternate path and generates an alert for the network administrator. Thus, the algorithm improves network reliability, routing efficiency, fault tolerance, and overall network performance by adapting to changing network conditions.